

Apunte 17 - Introducción a Seguridad Web

SEGURIDAD EN SITIOS WEB

Internet es un lugar peligroso. Con frecuencia escuchamos sobre sitios web que dejan de estar disponibles debido a ataques, o que filtran millones de contraseñas y datos personales. El propósito de la seguridad web es prevenir estos ataques: proteger sitios web del acceso, uso, modificación, destrucción o interrupción no autorizados.

La seguridad eficaz requiere esfuerzos en todas las capas de nuestra aplicación: frontend (React), backend (Express), base de datos (SQLite/Sequelize), configuración del servidor y políticas de acceso.

Nota para el stack del curso: Al construir aplicaciones con React + Express + Sequelize, estamos construyendo esencialmente una API REST consumida por un SPA. Esto implica que debemos pensar la seguridad tanto desde la perspectiva de una aplicación web como desde la de una API. Por eso en este apunte cubrimos dos marcos de referencia OWASP: el Top 10 general y el API Security Top 10.

OWASP

El **Open Web Application Security Project (OWASP)** es una fundación sin fines de lucro que trabaja para mejorar la seguridad del software. Publica documentos de referencia estándar sobre los riesgos de seguridad más críticos:

- **OWASP Top 10:2025** (aplicaciones web generales): <https://owasp.org/Top10/2025/>
- **OWASP API Security Top 10:2023** (APIs REST/GraphQL): <https://owasp.org/API-Security/>

En este apunte cubrimos ambos, ya que nuestro stack combina una aplicación web (React) con una API REST (Express).

PARTE 1: OWASP Top 10:2025

A01:2025 - Broken Access Control (Control de Acceso Roto)

Mantiene el puesto #1. El **100% de las aplicaciones testeadas** presentaron alguna forma de control de acceso roto. Abarca 40 CWEs y 1.839.701 ocurrencias detectadas.

El control de acceso impone políticas para que los usuarios no excedan sus permisos. Las vulnerabilidades comunes incluyen:

- Violar el principio de menor privilegio otorgando acceso excesivo por defecto
- Eludir controles mediante manipulación de URLs, parámetros o peticiones API directas
- Acceder a cuentas de otros usuarios mediante referencias directas inseguras (IDs predecibles)
- Falta de controles en métodos POST/PUT/DELETE de APIs
- Escalación de privilegios sin autenticación adecuada
- Manipulación de tokens, cookies o metadatos JWT
- Mala configuración de CORS que permite acceso no autorizado

Ejemplo vulnerable en Express:

```
// MAL: no verifica que la cuenta pertenezca al usuario
app.get('/api/cuentas/:id', authenticateToken, async (req, res) => {
  const cuenta = await Cuenta.findByPk(req.params.id);
  res.json(cuenta);
});
```

Ejemplo corregido:

```
// BIEN: verifica ownership del recurso
app.get('/api/cuentas/:id', authenticateToken, async (req, res) => {
  const cuenta = await Cuenta.findOne({
    where: { id: req.params.id, usuarioId: req.user.id }
  });
  if (!cuenta) return res.status(403).json({ message: 'Acceso denegado' });
  res.json(cuenta);
});
```

Prevención:

- Negar acceso por defecto para recursos no públicos
- Implementar un mecanismo centralizado de control de acceso para toda la aplicación
- Verificar ownership de registros en cada endpoint, no solo autenticación
- Usar JWTs de corta duración con patrón de refresh token
- Invalidar identificadores de sesión del lado del servidor al hacer logout
- Limitar tasa de llamadas a la API (`express-rate-limit`)
- Loguear y alertar ante fallos de acceso
- Deshabilitar listado de directorios y remover archivos de metadata (.git) de la raíz web
- Incluir tests de control de acceso en pruebas unitarias e integración

A02:2025 - Security Misconfiguration (Configuración Incorrecta de Seguridad)

Subió al puesto #2. Ocurre cuando un sistema, aplicación o servicio cloud está configurado incorrectamente desde la perspectiva de seguridad.

Se manifiesta cuando:

- Falta hardening de seguridad en cualquier capa del stack o permisos cloud mal configurados
- Features, puertos, servicios o cuentas por defecto innecesarios permanecen activos
- El manejo de errores expone stack traces sensibles al usuario
- Faltan cabeceras de seguridad o están mal configuradas
- Se prioriza compatibilidad hacia atrás sobre configuración segura

Protección con helmet.js (middleware esencial para Express):

```
import helmet from 'helmet';

app.use(helmet()); // Configura ~15 cabeceras de seguridad en una línea
```

Helmet configura automáticamente: **Content-Security-Policy**, **Strict-Transport-Security**, **X-Content-Type-Options**, **X-Frame-Options**, **Referrer-Policy**, entre otros.

Configuración correcta de CORS:

```
import cors from 'cors';

// MAL: permite cualquier origen
app.use(cors());

// BIEN: solo permite el origen del frontend
app.use(cors({
  origin: 'http://localhost:3000', // URL del frontend React
  credentials: true                // necesario si usamos cookies
}));
```

Prevención:

- Automatizar el despliegue para configuraciones idénticas entre dev, QA y producción (con credenciales diferentes por entorno)
- Plataforma mínima: remover features, componentes y aplicaciones de ejemplo no utilizadas
- Revisar y actualizar configuraciones de seguridad regularmente
- Usar federación de identidad y credenciales de corta duración en vez de embeber secretos en código
- Segmentar arquitectura con contenedores o grupos de seguridad (ACLs)

Escenarios de ataque:

1. Aplicaciones de ejemplo con fallas conocidas quedan en servidores de producción
2. Listado de directorios habilitado permite descargar código compilado
3. Mensajes de error verbosos exponen versiones de componentes con vulnerabilidades conocidas
4. Permisos de almacenamiento cloud por defecto permiten acceso público

A03:2025 - Software Supply Chain Failures (Fallas en la Cadena de Suministro de Software)

Categoría nueva en 2025. El 50% de los encuestados la clasificaron como su preocupación #1. Abarca todas las fallas en la cadena de suministro de software, no solo componentes desactualizados.

Las fallas en la cadena de suministro son quiebres o compromisos en el proceso de construir, distribuir o actualizar software. El riesgo proviene de vulnerabilidades o modificaciones maliciosas en código de terceros, herramientas y dependencias.

Indicadores de riesgo: La organización está en riesgo elevado cuando:

- No tiene un inventario completo de versiones de componentes (dependencias directas y transitivas)
- Usa software sin soporte o desactualizado
- No escanea vulnerabilidades regularmente
- No documenta cambios en pipelines CI/CD, repositorios e IDEs
- Obtiene componentes de orígenes no confiables
- Aplica parches con poca frecuencia

Casos reales:

- **SolarWinds (2019):** Software comprometido del proveedor afectó ~18.000 organizaciones
- **Log4Shell (2021):** CVE-2021-44228 en Log4j2 permitió ejecución remota de código
- **Bybit (2025):** Ataque a la cadena de suministro en software de wallet robó \$1.500 millones
- **Shai-Hulud (2025):** Primer gusano auto-propagante en npm — paquetes maliciosos recolectaban datos y se propagaban automáticamente usando tokens npm robados

En nuestro stack Node.js:

```
# Auditar dependencias
npm audit

# Instalar de forma reproducible (usa lockfile estrictamente)
npm ci

# Verificar dependencias antes de instalar
npm install --dry-run nombre-paquete
```

Prevención:

- Mantener un Software Bill of Materials (SBOM) usando herramientas como OWASP Dependency-Track
- Monitorear continuamente fuentes como CVE, NVD y OSV para vulnerabilidades
- Obtener componentes exclusivamente de fuentes oficiales con paquetes firmados
- Elegir deliberadamente qué versión de una dependencia usar y actualizar solo cuando sea necesario
- Trackear modificaciones a CI/CD, repositorios e IDEs con control de versiones y pull requests
- Habilitar MFA en repositorios y servidores de build
- Usar despliegues staged o canary en vez de actualizaciones simultáneas

A04:2025 - Cryptographic Failures (Fallas Criptográficas)

Vulnerabilidades producidas por uso incorrecto de criptografía o algoritmos débiles/defectuosos.

- Almacenamiento de contraseñas en texto plano o con hashes débiles (MD5, SHA-1)
- Transmisión de datos sin HTTPS
- Claves de cifrado hardcodedas en el código fuente
- Uso de algoritmos criptográficos obsoletos

En nuestro stack: Nunca almacenar contraseñas sin hashear. Usar **bcrypt** (mínimo) o **Argon2id** (recomendado por OWASP):

```
import bcrypt from 'bcryptjs';

// Al registrar usuario
const salt = await bcrypt.genSalt(12);
const hashedPassword = await bcrypt.hash(password, salt);

// Al hacer login
const isValid = await bcrypt.compare(passwordIngresada, user.password);
```

Prevención:

- Usar HTTPS para toda comunicación (configurar HSTS)
- No almacenar datos sensibles innecesariamente
- Usar algoritmos actualizados y bien establecidos
- Almacenar secretos en variables de entorno, nunca en código fuente

A05:2025 - Injection (Inyección)

Ocurre cuando datos no confiables del usuario se envían a un intérprete (navegador, base de datos, línea de comandos) y causan que el intérprete ejecute partes de esa entrada como comandos.

Incluye: SQL, NoSQL, OS command, ORM, LDAP, XSS y Expression Language injection.

La aplicación es vulnerable cuando:

- La entrada del usuario no se valida, filtra o sanitiza
- Se usan consultas dinámicas sin parametrizar
- Datos sin sanitizar aparecen en parámetros de búsqueda del ORM
- Datos hostiles se concatenan directamente en queries o comandos

SQL Injection:

```
-- Entrada maliciosa: ' OR '1'='1
SELECT * FROM users WHERE name = '' OR '1'='1';
-- Retorna TODOS los registros de la base de datos
```

Protección con Sequelize (nuestro ORM): Sequelize parametriza las consultas automáticamente:

```
// SEGURO: Sequelize parametriza automáticamente
const user = await Usuario.findOne({ where: { username: req.body.username } });

// PELIGROSO: query raw sin parametrizar
const [results] = await sequelize.query(
  `SELECT * FROM users WHERE name = '${req.body.name}'` // NUNCA hacer esto
);

// SEGURO: query raw parametrizada
```

```
const [results] = await sequelize.query(
  'SELECT * FROM users WHERE name = ?',
  { replacements: [req.body.name] }
);
```

Command Injection:

```
// Si un endpoint recibe un dominio para hacer ping:
// Entrada maliciosa: example.com; cat /etc/passwd
// Ejecuta el comando arbitrario después del ;
```

XSS (Cross-Site Scripting) - Un atacante inyecta scripts que se ejecutan en el navegador de otros usuarios:

```
// SEGURO: React escapa automáticamente
const Comentario = ({ texto }) => <p>{texto}</p>;
// Si texto = "<script>alert('xss')</script>", React lo renderiza como texto plano

// PELIGROSO: bypass del auto-escaping
<div dangerouslySetInnerHTML={{ __html: contenidoDelUsuario }} />
// NUNCA usar dangerouslySetInnerHTML con datos del usuario
```

Validación en Express con **express-validator**:

```
import { body, validationResult } from 'express-validator';

app.post('/api/articulos',
  body('titulo').trim().escape().isLength({ min: 1, max: 100 }),
  body('precio').isFloat({ min: 0 }),
  async (req, res) => {
    const errors = validationResult(req);
    if (!errors.isEmpty()) return res.status(400).json({ errors: errors.array()
  });
  // continuar...
}
);
```

Prevención:

- Usar APIs seguras que eviten el intérprete directamente y proporcionen interfaces parametrizadas
- Implementar validación de entrada positiva del lado del servidor
- Escapar caracteres especiales usando la sintaxis específica del intérprete
- Combinar revisión de código con testing automatizado (SAST/DAST) en pipelines CI/CD

Se origina en la etapa de diseño y arquitectura. No es un problema de implementación sino de falta de controles de seguridad desde el diseño.

- No considerar threat modeling (modelado de amenazas)
- No implementar límites de negocio (ej: sin límite de intentos de login)
- No separar lógica de autenticación de lógica de negocio
- No considerar escenarios de abuso desde la fase de diseño

A07:2025 - Authentication Failures (Fallas de Autenticación)

Mecanismos de autenticación mal implementados: contraseñas débiles, transmisión insegura de credenciales, falta de autenticación multifactor, sesiones mal gestionadas.

Este tema se profundiza en el **Apunte 18 - Autenticación y Autorización**.

A08:2025 - Software or Data Integrity Failures (Fallas de Integridad de Software o Datos)

Datos o software modificados malintencionadamente o accidentalmente. Incluye:

- Actualizaciones de software sin verificar firma
- Pipelines de CI/CD inseguros
- Deserialización insegura

En nuestro stack: Usar lockfiles (`package-lock.json`) y `npm ci` en producción para garantizar que se instalen exactamente las versiones verificadas.

A09:2025 - Security Logging and Alerting Failures (Fallas en Registro y Alertas de Seguridad)

Falta de mecanismos adecuados de logging y monitoreo. Sin logs no podemos detectar ni responder a incidentes de seguridad.

Recomendaciones:

- Loggear intentos de login fallidos
 - Loggear accesos denegados
 - **No loggear datos sensibles** (contraseñas, tokens, datos de tarjetas)
 - Implementar alertas ante patrones sospechosos (múltiples fallos de login, accesos inusuales)
-

A10:2025 - Mishandling of Exceptional Conditions (Manejo Incorrecto de Condiciones Excepcionales)

Categoría nueva en 2025. Combina 24 CWEs con 769.581 ocurrencias detectadas. Ocurre cuando las aplicaciones fallan en prevenir, detectar y responder a situaciones inusuales e impredecibles.

Tres fallas fundamentales:

1. **Falla de prevención:** No poder evitar situaciones inusuales
2. **Falla de detección:** No detectar la situación cuando ocurre

3. **Falla de respuesta:** Manejo pobre o ausente después del hecho

Causas comunes: validación de entrada incompleta, manejo de errores tardío a nivel alto, estados ambientales inesperados y manejo inconsistente de excepciones.

Escenarios de ataque:

1. **DoS por agotamiento de recursos:** Excepciones no manejadas en carga de archivos no liberan recursos, consumiendo toda la capacidad disponible.
2. **Exposición de datos:** Errores de base de datos revelan información sensible del sistema que permite reconocimiento para SQL injection.
3. **Fraude financiero:** Rollback incompleto de transacciones en procesos multi-paso permite a atacantes drenar cuentas o duplicar transferencias mediante interrupciones de red.

Prevención:

- Capturar errores directamente donde ocurren con manejo significativo
- Implementar logging y alertas comprehensivos
- Desplegar un manejador global de excepciones como red de seguridad
- Usar rate limiting, cuotas de recursos y throttling para prevenir condiciones excepcionales
- Aplicar validación estricta de entrada con centralización

Regla crítica: Si estás a mitad de una transacción de cualquier tipo, es extremadamente importante hacer rollback de toda la transacción y empezar de nuevo (también conocido como "fail closed").

```
// Ejemplo de manejo correcto en Express
app.post('/api/transferencia', authenticateToken, async (req, res) => {
  const transaction = await sequelize.transaction();
  try {
    await CuentaOrigen.decrement('saldo', { by: monto, transaction });
    await CuentaDestino.increment('saldo', { by: monto, transaction });
    await transaction.commit();
    res.json({ message: 'Transferencia exitosa' });
  } catch (error) {
    await transaction.rollback(); // SIEMPRE hacer rollback si algo falla
    // Loguear el error internamente, NO exponer detalles al usuario
    console.error('Error en transferencia:', error.message);
    res.status(500).json({ message: 'Error al procesar la transferencia' });
  }
});
```

PARTE 2: OWASP API Security Top 10:2023

Dado que nuestro stack React + Express construye esencialmente una API REST, es fundamental conocer las amenazas específicas para APIs.

Referencia: <https://owasp.org/API-Security/editions/2023/en/0x11-t10/>

API1:2023 - Broken Object Level Authorization (BOLA)

Fallas de autorización cuando las APIs exponen endpoints que usan identificadores de objetos. Se requieren verificaciones de control de acceso en cada función que accede a datos mediante IDs proporcionados por el usuario.

similiar a: A01:2025 - Broken Access Control (Control de Acceso Roto)

API2:2023 - Broken Authentication

Implementación incorrecta de autenticación que permite a atacantes comprometer tokens o asumir identidades de otros usuarios.

Se profundiza en el siguiente apunte.

API3:2023 - Broken Object Property Level Authorization

Combina exposición excesiva de datos y asignación masiva (mass assignment). Falta de validación de autorización a nivel de propiedades del objeto.

```
// VULNERABLE: expone todas las propiedades, incluyendo password hasheado
app.get('/api/usuarios/:id', authenticateToken, async (req, res) => {
  const usuario = await Usuario.findByPk(req.params.id);
  res.json(usuario); // Envía TODO incluyendo password, role, etc.
});

// SEGURO: selecciona solo las propiedades necesarias
app.get('/api/usuarios/:id', authenticateToken, async (req, res) => {
  const usuario = await Usuario.findByPk(req.params.id, {
    attributes: ['id', 'username', 'email'] // Solo campos públicos
  });
  res.json(usuario);
});
```

Mass Assignment:

```
// VULNERABLE: pasa todo el body al modelo
const articulo = await Artículo.create(req.body);
// Si el atacante envía { titulo: "...", precio: 0, usuarioId: 999, role: "admin"
}

// SEGURO: whitelist de campos permitidos
const { titulo, precio, stock } = req.body;
const articulo = await Artículo.create({ titulo, precio, stock, usuarioId:
req.user.id });
```

API4:2023 - Unrestricted Resource Consumption

Agotamiento de recursos computacionales (ancho de banda, CPU, memoria) y servicios pagos vía API. Puede causar denegación de servicio o incremento de costos operativos.

```
import rateLimit from 'express-rate-limit';

// Limitar peticiones generales
const generalLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 100
});

// Limitar login más estrictamente
const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5,
  message: { error: 'Demasiados intentos, intente en 15 minutos' }
});

app.use('/api/', generalLimiter);
app.post('/api/login', loginLimiter, loginController);
```

API5:2023 - Broken Function Level Authorization

Fallas de autorización en jerarquías complejas de control de acceso. Separación poco clara entre funciones administrativas y regulares.

```
// Middleware de autorización por roles
function authorizeRole(...rolesPermitidos) {
  return (req, res, next) => {
    if (!rolesPermitidos.includes(req.user.role)) {
      return res.status(403).json({ message: 'No tiene permisos para esta acción' });
    }
    next();
  };
}

// Solo admin puede eliminar
app.delete('/api/articulos/:id', authenticateToken, authorizeRole('admin'),
deleteArticulo);
```

API6:2023 - Unrestricted Access to Sensitive Business Flows

APIs que exponen flujos de negocio sin protección contra explotación automatizada (bots que compran todo el stock, crean cuentas masivamente, etc.).

API7:2023 - Server Side Request Forgery (SSRF)

La API obtiene recursos remotos sin validar las URIs proporcionadas por el usuario:

```
// VULNERABLE: el usuario controla la URL
app.get('/api/fetch-url', async (req, res) => {
  const response = await fetch(req.query.url); // PELIGROSO
  res.json(await response.json());
});

// El atacante podría enviar: ?url=http://169.254.169.254/latest/meta-data/
// (accediendo a metadatos internos del servidor cloud)
```

Mitigación: Validar y restringir URLs con listas blancas de dominios permitidos.

API8:2023 - Security Misconfiguration

Configuración impropia de APIs y sistemas que descuida mejores prácticas de seguridad. (Ver A02:2025 para detalles)

API9:2023 - Improper Inventory Management

Documentación y tracking inadecuado de endpoints, versiones y hosts de la API. Riesgo de exponer endpoints deprecados o de debug.

Ejemplo: Dejar habilitado un endpoint `/api/debug/users` que lista todos los usuarios sin autenticación en producción.

API10:2023 - Unsafe Consumption of APIs

Seguridad debilitada cuando los desarrolladores confían en datos de APIs de terceros más que en la entrada del usuario, creando vulnerabilidades a través de integraciones comprometidas.

PARTE 3: Otras amenazas relevantes

CSRF (Cross-Site Request Forgery)

Un atacante puede ejecutar acciones usando las credenciales de otro usuario sin su conocimiento.

Mitigación moderna: El uso de `SameSite=Strict` en cookies junto con el patrón Bearer token en cabecera `Authorization` mitiga la mayoría de ataques CSRF. Si se usa autenticación solo por cookies, agregar tokens CSRF con paquetes como `csrf-csrf`.

Clickjacking

Un atacante coloca un sitio legítimo en un iframe invisible y captura clicks del usuario. **Helmet.js lo previene** configurando `X-Frame-Options` automáticamente.

Denegación de Servicio (DoS)

Inundar el servidor con peticiones para interrumpir el servicio. Mitigado con `express-rate-limit` (ver API4:2023).

PARTE 4: Seguridad específica para React (Frontend)

- 1. **JSX auto-escaping:** React escapa automáticamente el contenido en JSX. No usar `dangerouslySetInnerHTML` con datos del usuario.
- 2. **Variables de entorno:** Solo las variables con prefijo `VITE_` (Vite) se incluyen en el bundle del cliente. **Nunca poner secretos** (API keys privadas, claves JWT) en variables de entorno del frontend — todo lo que va al bundle es visible para cualquier usuario.
- 3. **Subresource Integrity (SRI):** Si se cargan scripts desde CDN, usar atributos `integrity` para verificar que no fueron modificados.
- 4. **Prototype pollution:** Vulnerabilidad en dependencias JavaScript donde un atacante modifica el prototipo de objetos base. Mantener dependencias actualizadas.

Resumen: Medidas concretas para nuestro stack

Capa	Herramienta/Práctica	Propósito
Express	<code>helmet</code>	Cabeceras de seguridad (CSP, HSTS, X-Frame-Options)
Express	<code>cors</code>	Controlar orígenes permitidos
Express	<code>express-rate-limit</code>	Prevenir fuerza bruta y DoS
Express	<code>express-validator</code>	Validar y sanitizar inputs
Base de datos	Sequelize (ORM)	Queries parametrizadas (previene SQL injection)
Contraseñas	<code>bcryptjs</code> o <code>argon2</code>	Hasheo seguro de contraseñas
Auth	<code>jsonwebtoken</code>	Tokens JWT para autenticación
React	JSX auto-escaping	Previene XSS
Dependencias	<code>npm audit</code> / <code>npm ci</code>	Detectar vulnerabilidades, instalar reproducible
HTTPS	Certificados SSL/TLS	Encriptar comunicación
Errores	try/catch + transaction rollback	Manejo correcto de condiciones excepcionales

Ejercicio propuesto

Ejercicio Seguridad: Sobre la aplicación de Artículos desarrollada en semanas anteriores:

- 1. Instalar y configurar `helmet` y `cors` con opciones restrictivas
- 2. Agregar `express-rate-limit` en la ruta de login (máximo 5 intentos en 15 minutos)
- 3. Agregar validación con `express-validator` en las rutas POST/PUT de artículos
- 4. Verificar con `npm audit` que no haya vulnerabilidades en las dependencias

5. Identificar en el código algún ejemplo de Mass Assignment y corregirlo usando whitelist de campos
6. Verificar que ningún endpoint exponga campos sensibles del modelo Usuario (password, etc.) — aplicar `attributes` en las consultas Sequelize
7. Agregar manejo de errores con try/catch en al menos una ruta que use transacciones